

1. Problema del Statement

El uso de `Statement` tiene varios inconvenientes:

Problemas principales

- Construcción de consultas concatenando strings
- Código difícil de mantener
- Bajo rendimiento en ejecuciones repetidas
- Vulnerable a ataques de inyección SQL

Ejemplo peligroso

```
String sql = "SELECT * FROM usuarios WHERE nombre = " + nombre + """;  
Statement stmt = conn.createStatement();  
ResultSet rs = stmt.executeQuery(sql);
```

Si el usuario introduce:

```
' OR '1'='1
```

La consulta se rompe y devuelve todos los registros.

2. Qué es PreparedStatement

`PreparedStatement` es una versión mejorada de `Statement`.

Características

- Consultas precompiladas
 - Uso de parámetros (?)
 - Más eficiente
 - Más seguro
-

3. Sintaxis básica

```
String sql = "SELECT * FROM usuarios WHERE nombre = ?";
```

```
PreparedStatement ps = conn.prepareStatement(sql);
```

```
ps.setString(1, "Juan");
```

```
ResultSet rs = ps.executeQuery();
```

4. Parámetros dinámicos

Los `?` se sustituyen con métodos `setX()`.

Ejemplos

```
ps.setString(1, "Juan");
```

```
ps.setInt(2, 10);
```

```
ps.setDouble(3, 50.5);
```

Reglas

- Empiezan en índice 1 (no 0)
 - Deben coincidir con el tipo de dato
-

5. Ejemplo con SELECT

```
import java.sql.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            Connection conn = DriverManager.getConnection(
```

```
                "jdbc:mysql://localhost:3306/prueba",
```

```
                "root",
```

```
                "1234"
```

```
            );
```

```
            String sql = "SELECT * FROM usuarios WHERE id = ?";
```

```
            PreparedStatement ps = conn.prepareStatement(sql);
```

```
            ps.setInt(1, 1);
```

```
            ResultSet rs = ps.executeQuery();
```

```
        while (rs.next()) {
            System.out.println(rs.getString("nombre"));
        }

        conn.close();

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}
```

6. Ejemplo con INSERT

```
String sql = "INSERT INTO usuarios (id, nombre) VALUES (?, ?)";
```

```
PreparedStatement ps = conn.prepareStatement(sql);
```

```
ps.setInt(1, 5);
ps.setString(2, "Mario");
```

```
ps.executeUpdate();
```

7. Ejemplo con UPDATE

```
String sql = "UPDATE usuarios SET nombre = ? WHERE id = ?";
```

```
PreparedStatement ps = conn.prepareStatement(sql);
```

```
ps.setString(1, "Lucía");
ps.setInt(2, 1);
```

```
ps.executeUpdate();
```

8. Ejemplo con DELETE

```
String sql = "DELETE FROM usuarios WHERE id = ?";
```

```
PreparedStatement ps = conn.prepareStatement(sql);
```

```
ps.setInt(1, 3);
```

```
ps.executeUpdate();
```

9. Ventajas frente a Statement

Seguridad

Evita inyección SQL porque no concatena strings.

Rendimiento

La consulta se precompila y se reutiliza.

Mantenimiento

Código más limpio y fácil de leer.

10. Buenas prácticas

- Usar siempre `PreparedStatement` en lugar de `Statement`
 - No concatenar datos del usuario en SQL
 - Usar tipos correctos (`setInt`, `setString`, etc.)
 - Reutilizar consultas cuando sea posible
-

11. Error común de alumnos

- Olvidar asignar todos los parámetros
- Usar índices incorrectos (empiezan en 1)
- No coincidir tipos de datos
- Pensar que `PreparedStatement` ejecuta automáticamente (siempre hay que llamar a `execute...()`)

EJERCICIOS

1. Insertar un usuario usando `PreparedStatement`.
2. Insertar varios usuarios cambiando los valores dinámicamente.
3. Hacer un SELECT con un parámetro (`WHERE id = ?`).
4. Actualizar el nombre de un usuario usando parámetros.
5. Eliminar un usuario usando `PreparedStatement`.
6. Crear una consulta que busque por nombre introducido por teclado.
7. Simular un ataque SQL Injection con `Statement` y luego corregirlo con `PreparedStatement`.
8. Reutilizar un mismo `PreparedStatement` cambiando parámetros.
9. Insertar 5 registros usando un bucle.
10. Validar que todos los parámetros han sido asignados antes de ejecutar.
11. Crear un sistema de login:
 - a. Pedir usuario y contraseña
 - b. Validar en BD con `PreparedStatement`
12. Insertar múltiples registros usando un bucle y un único `PreparedStatement`.
13. Crear un buscador:
 - a. Buscar productos por nombre parcial (`LIKE ?`)
14. Actualizar varios campos en una tabla usando un solo `PreparedStatement`.
15. Crear un método reutilizable:

`insertarUsuario(String nombre, int edad)`

Que use `PreparedStatement`.